

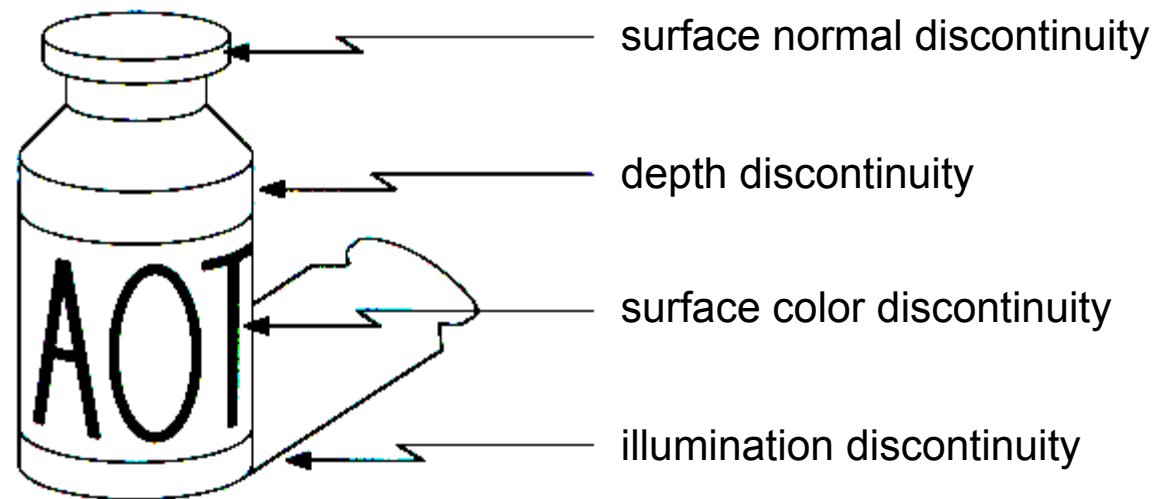
Edge detection

- **Goal:** Identify sudden changes (discontinuities) in an image
 - Intuitively, most semantic and shape information from the image can be encoded in the edges
 - More compact than pixels
- **Ideal:** artist's line drawing (but artist is also using object-level knowledge)



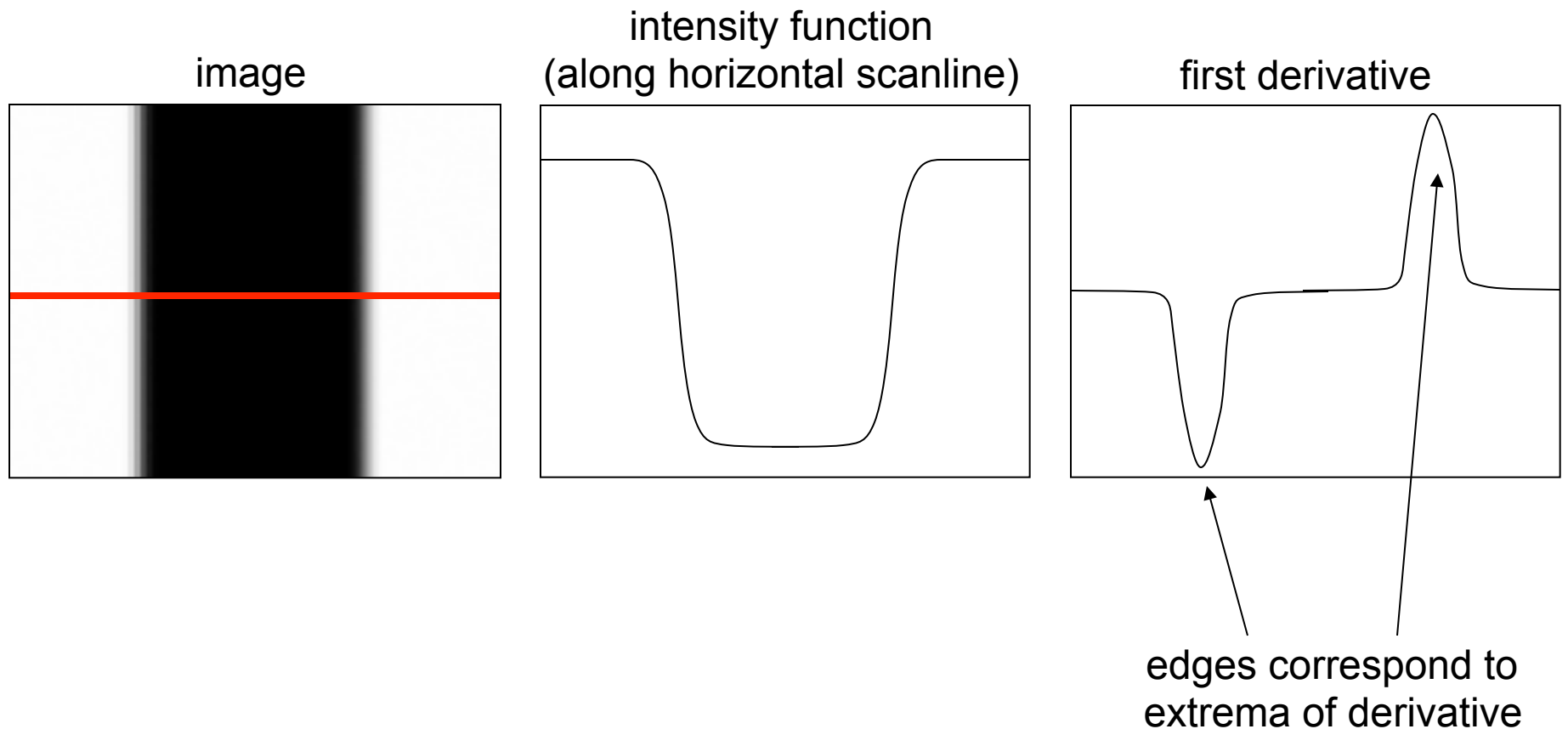
Origin of edges

Edges are caused by a variety of factors:



Characterizing edges

- An edge is a place of rapid change in the image intensity function



Derivatives with convolution

For 2D function $f(x,y)$, the partial derivative is:

$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

For discrete data, we can approximate using finite differences:

$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

To implement above as convolution, what would be the associated filter?

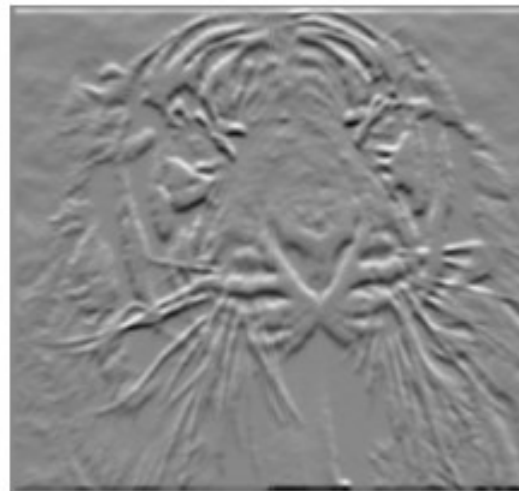
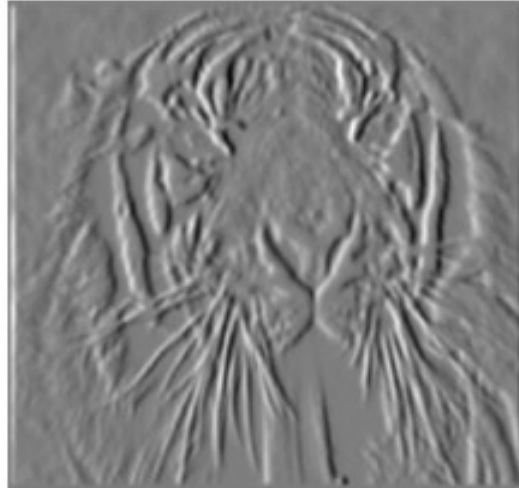
Partial derivatives of an image



$$\frac{\partial f(x, y)}{\partial x}$$

$$\frac{\partial f(x, y)}{\partial y}$$

-1	1
----	---



-1	or	1
1		-1

Which shows changes with respect to x?

Finite difference filters

Other approximations of derivative filters exist:

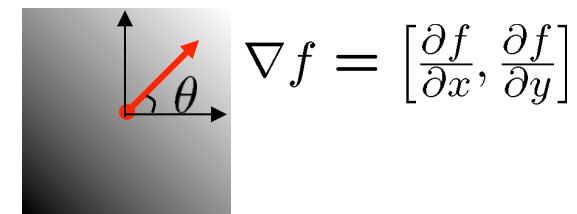
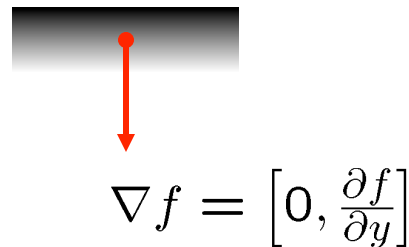
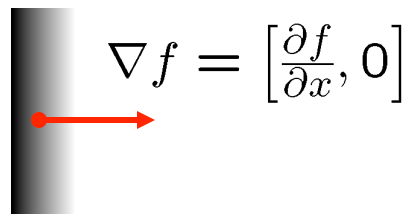
Prewitt: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} ; M_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$

Sobel: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} ; M_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$

Roberts: $M_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} ; M_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Image gradient

The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$



The gradient points in the direction of most rapid increase in intensity

- How does this direction relate to the direction of the edge?

The gradient direction is given by $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

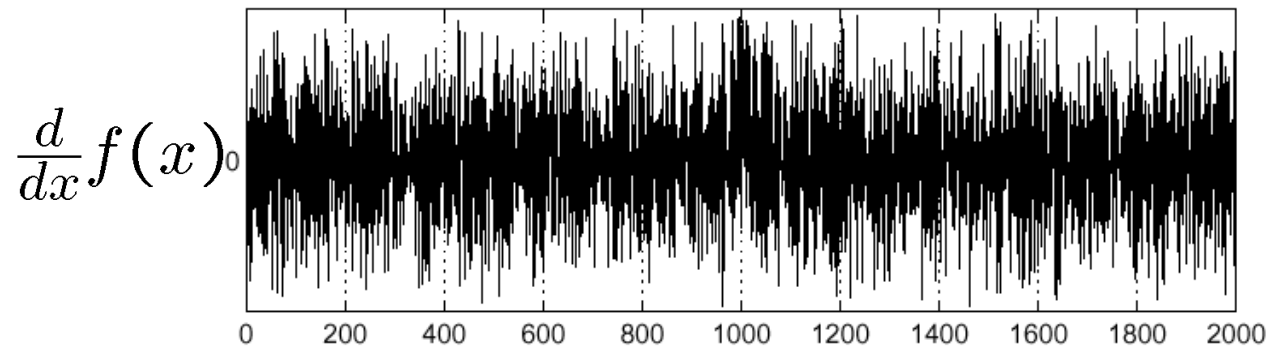
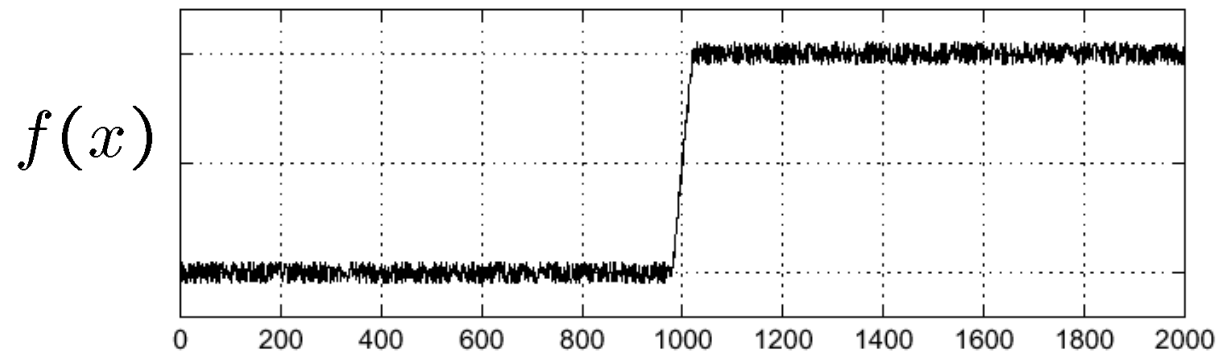
The *edge strength* is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Effects of noise

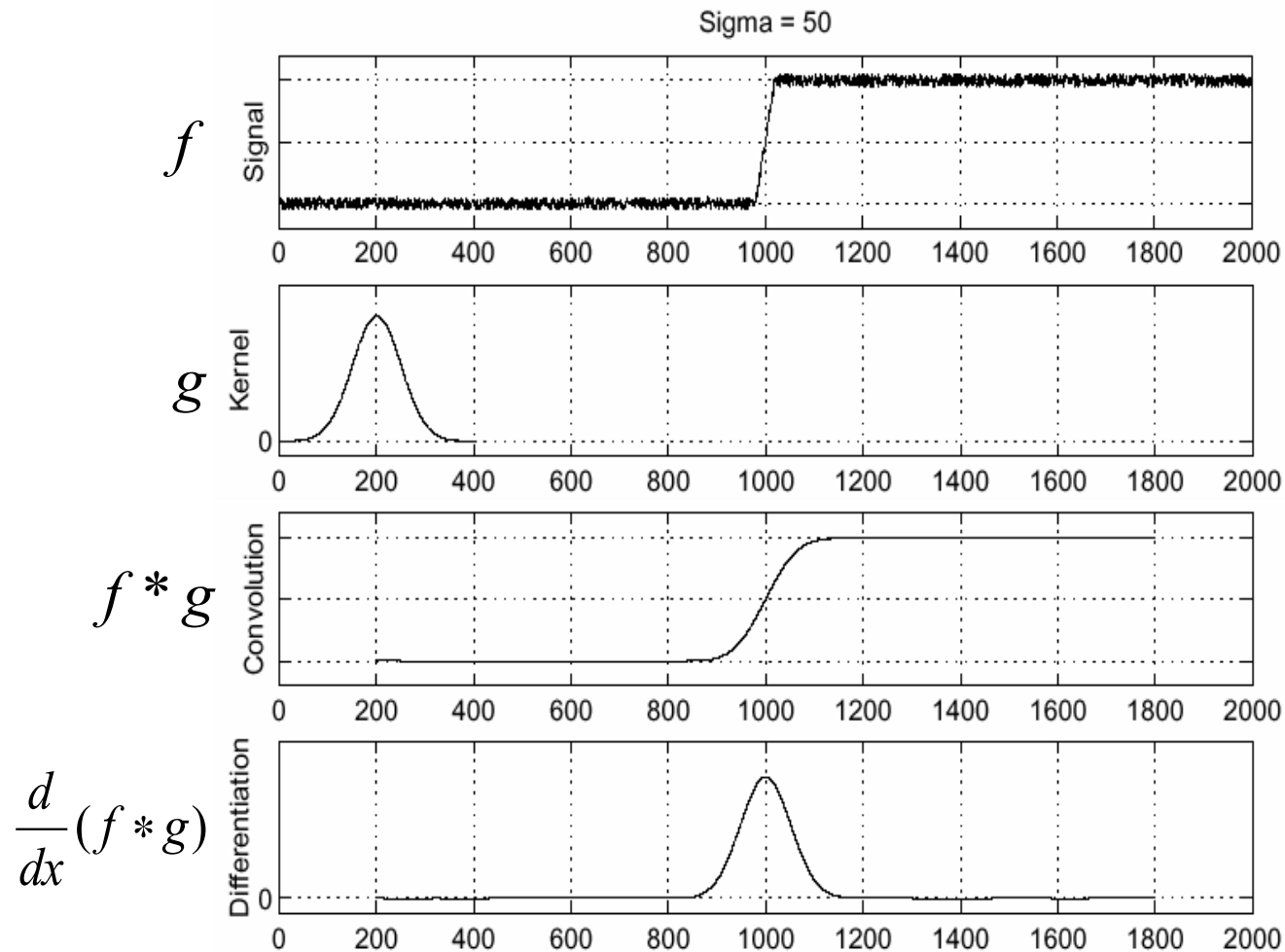
Consider a single row or column of the image

- Plotting intensity as a function of position gives a signal



Where is the edge?

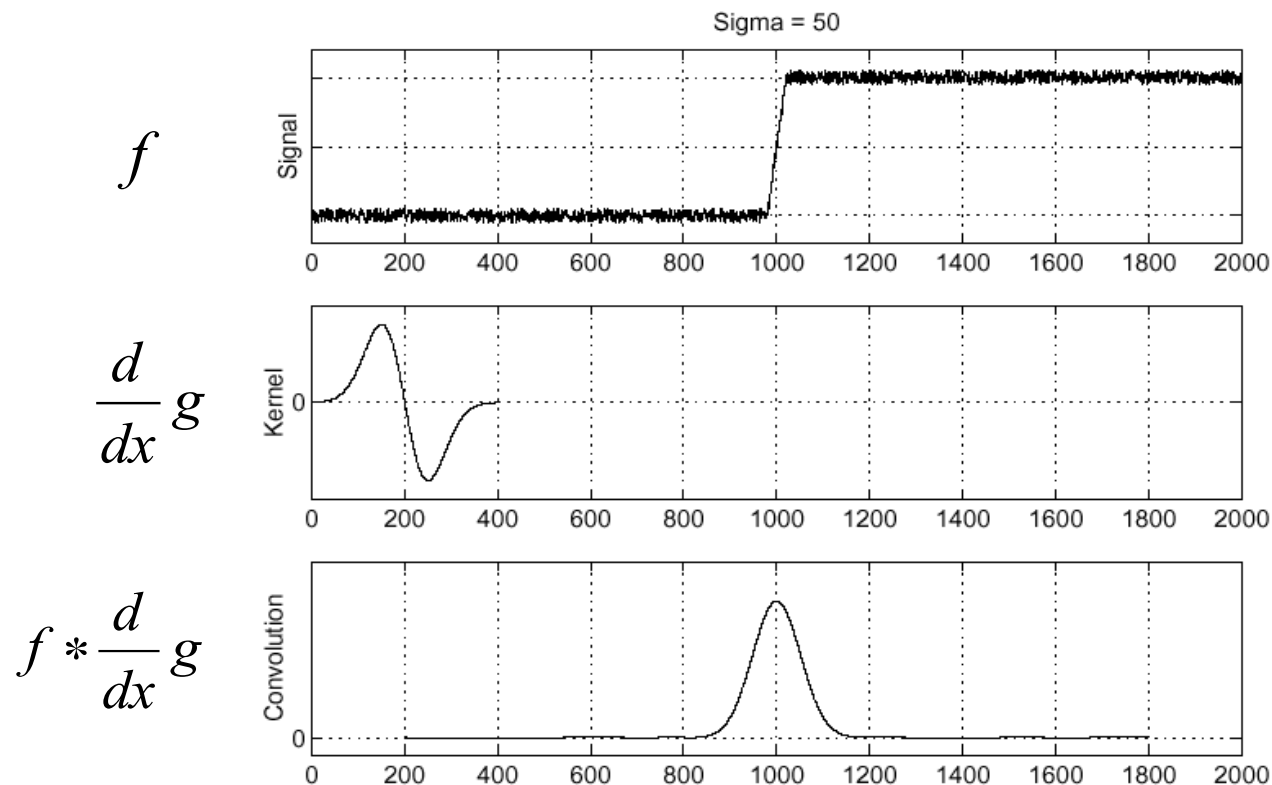
Solution: smooth first



- To find edges, look for peaks in $\frac{d}{dx}(f * g)$

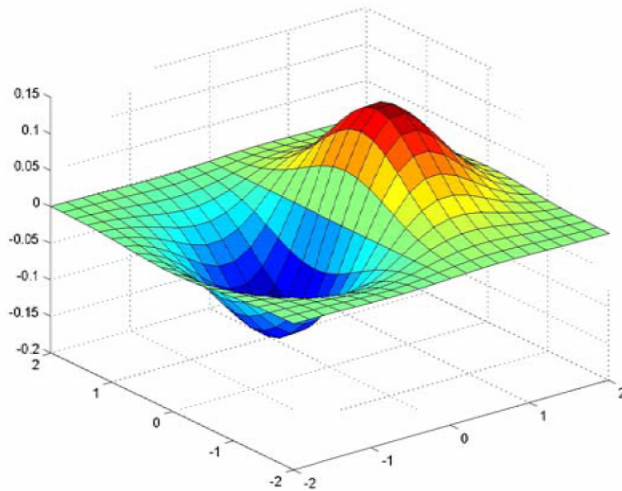
Derivative theorem of convolution

- Differentiation is convolution, and convolution is associative: $\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$
- This saves us one operation:

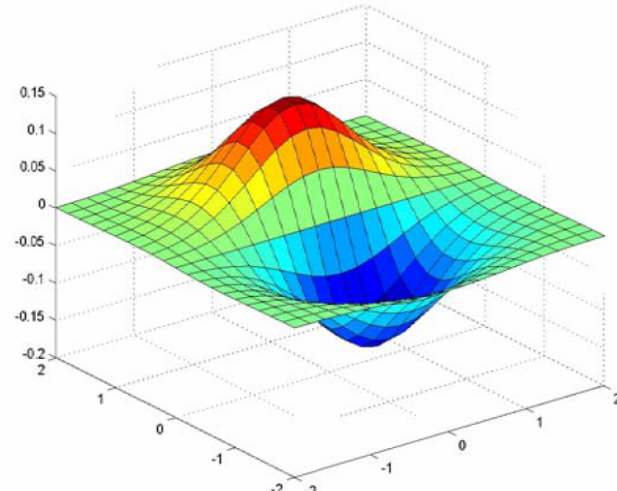


Source: S. Seitz

Derivative of Gaussian filter



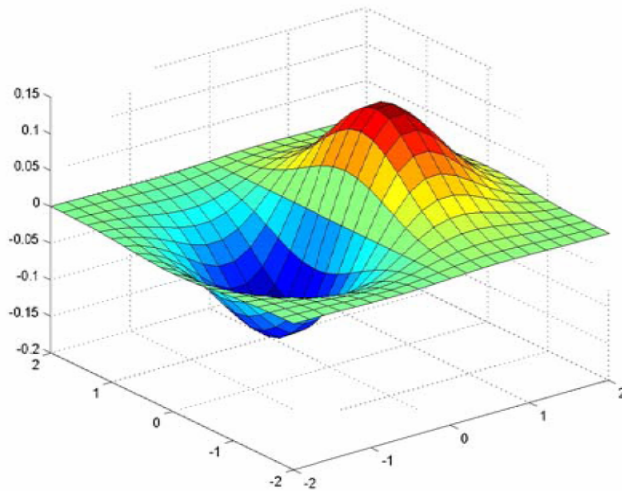
x-direction



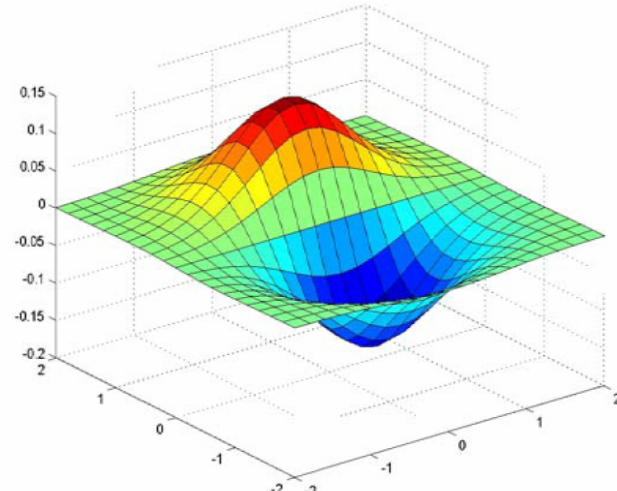
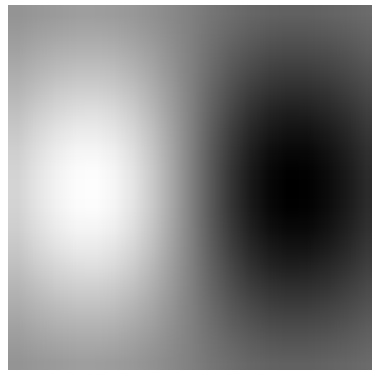
y-direction

Are these filters separable?

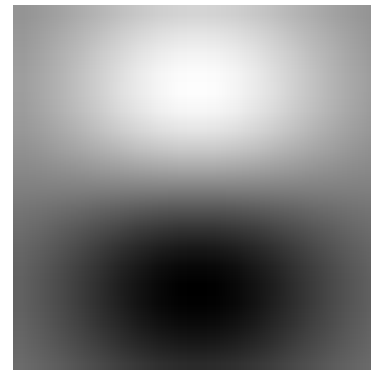
Derivative of Gaussian filter



x-direction

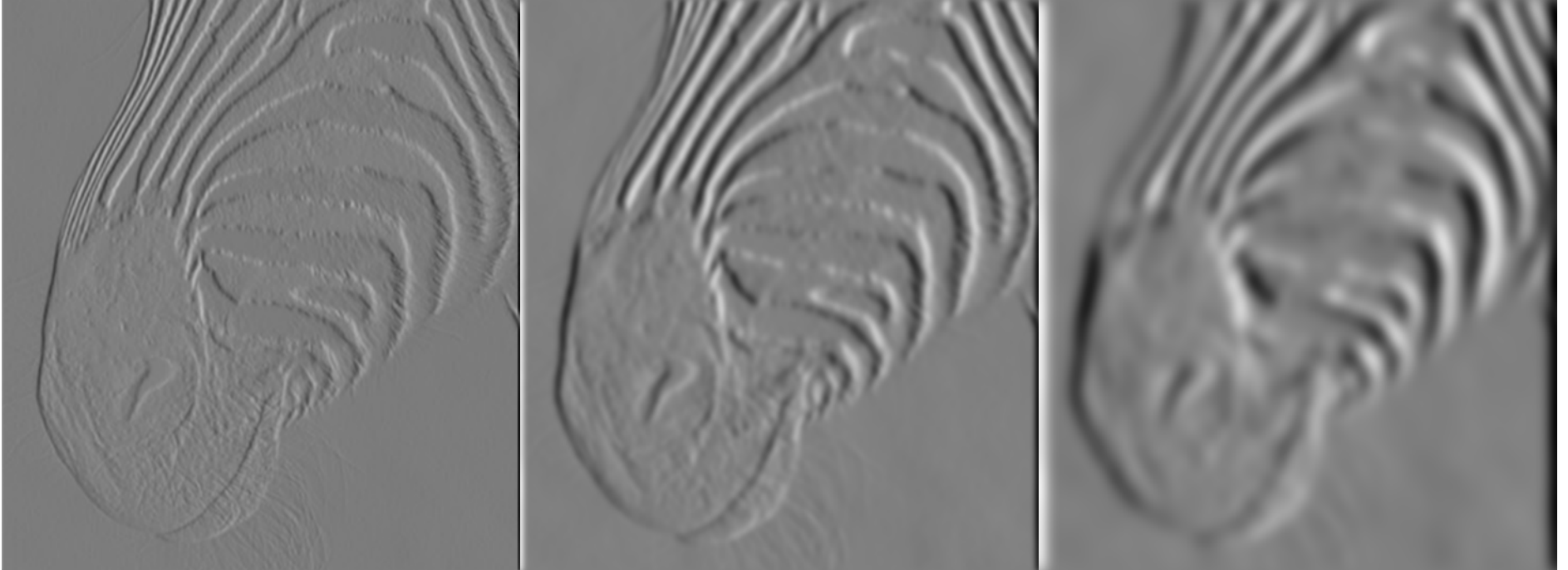


y-direction



Which one finds horizontal/vertical edges?

Scale of Gaussian derivative filter



1 pixel

3 pixels

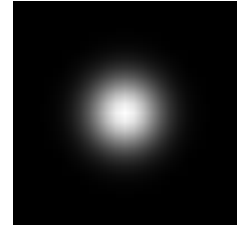
7 pixels

Smoothed derivative removes noise, but blurs edge. Also finds edges at different “scales”

Review: Smoothing vs. derivative filters

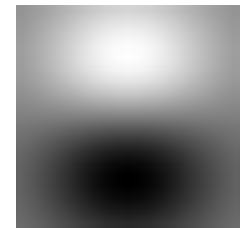
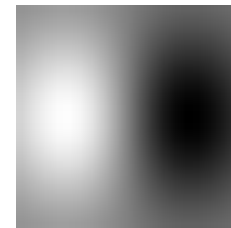
Smoothing filters

- Gaussian: remove “high-frequency” components; “low-pass” filter
- Can the values of a smoothing filter be negative?
- What should the values sum to?
 - **One**: constant regions are not affected by the filter



Derivative filters

- Derivatives of Gaussian
- Can the values of a derivative filter be negative?
- What should the values sum to?
 - **Zero**: no response in constant regions
- High absolute value at points of high contrast



The Canny edge detector



original image

The Canny edge detector



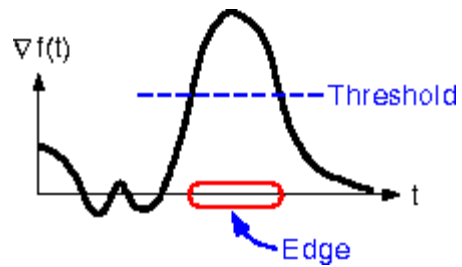
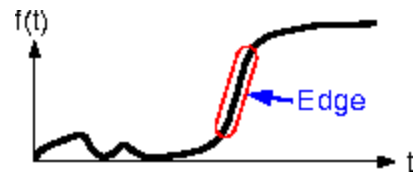
norm of the gradient

The Canny edge detector



thresholding

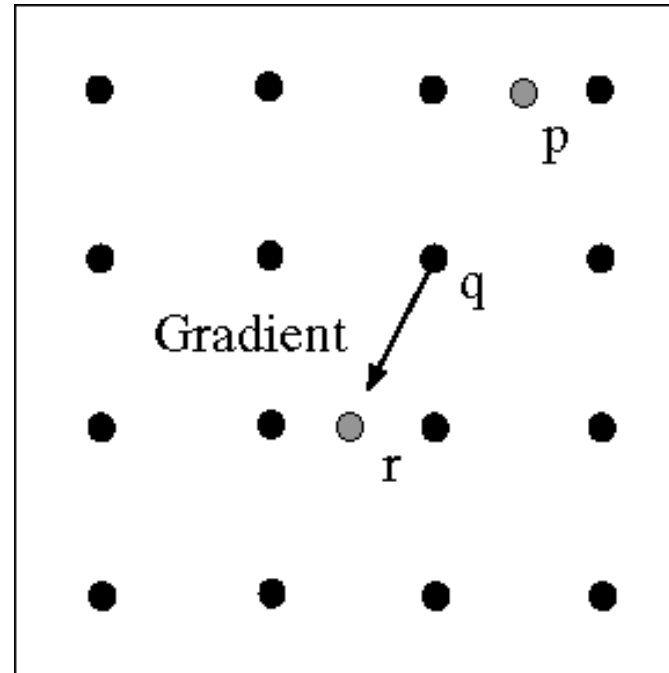
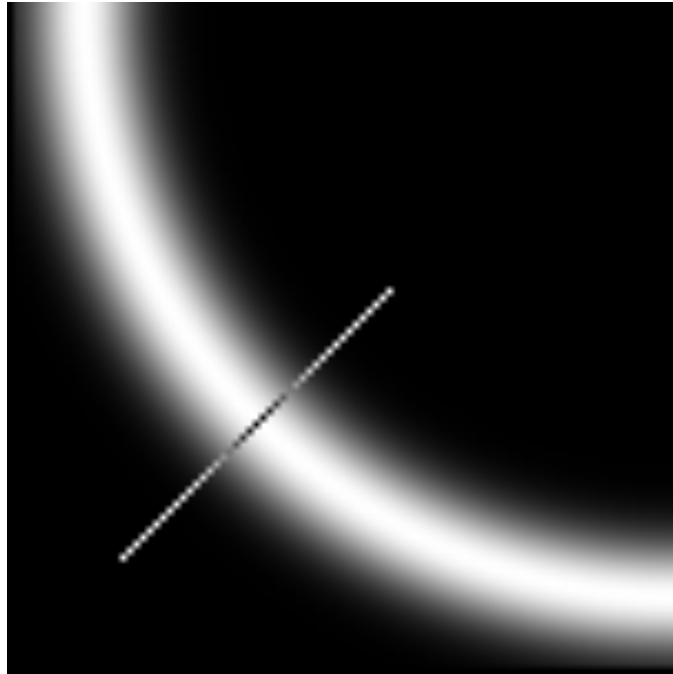
The Canny edge detector



How to turn these thick regions of the gradient into curves?

thresholding

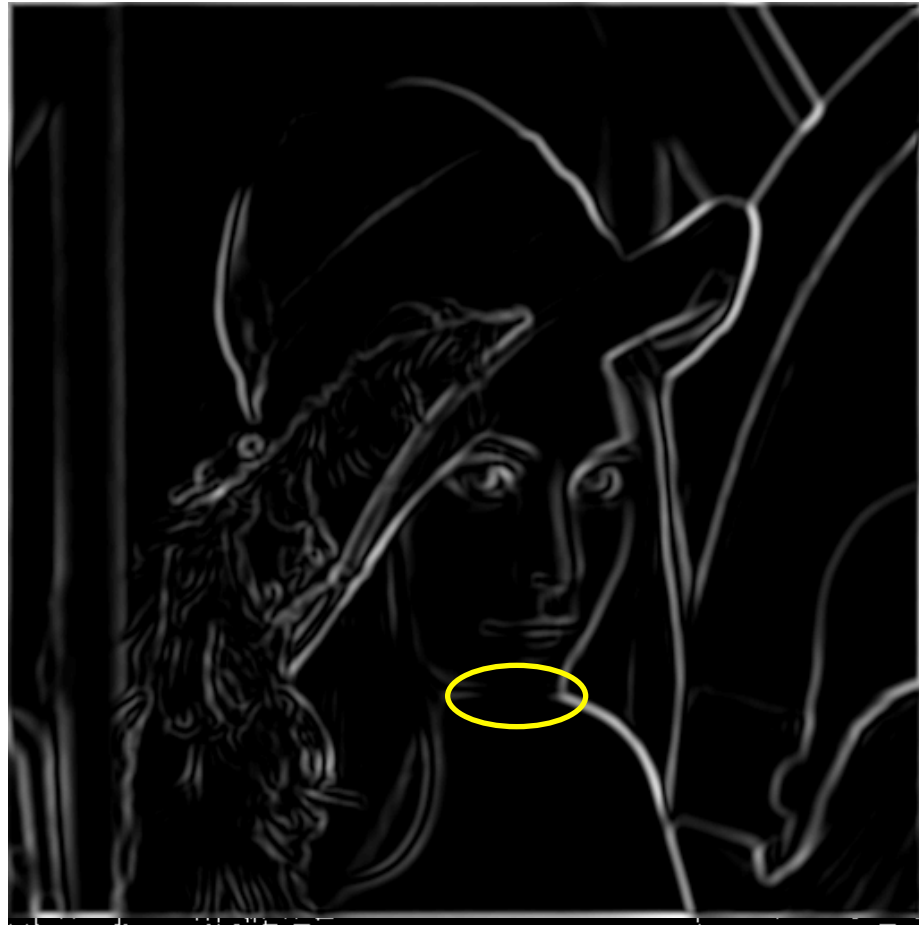
Non-maximum suppression



Check if pixel is local maximum along gradient direction, select single max across width of the edge

- requires checking interpolated pixels p and r

The Canny edge detector

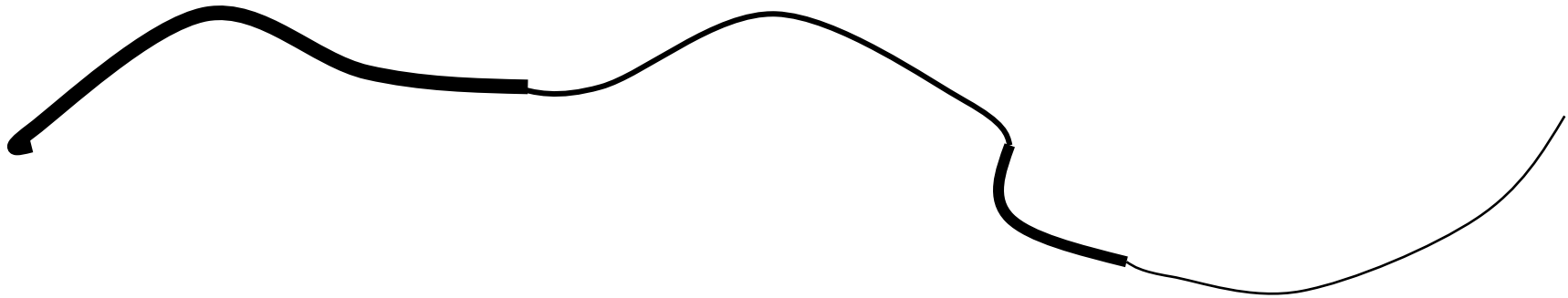


Problem:
pixels along
this edge
didn't
survive the
thresholding

thinning
(non-maximum suppression)

Hysteresis thresholding

Use a high threshold to start edge curves, and a low threshold to continue them.



Hysteresis thresholding



original image



**high threshold
(strong edges)**



**low threshold
(weak edges)**



hysteresis threshold

Recap: Canny edge detector

1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient
3. **Non-maximum suppression:**
 - Thin wide “ridges” down to single pixel width
4. **Linking and thresholding (hysteresis):**
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them

MATLAB: `edge(image, 'canny');`

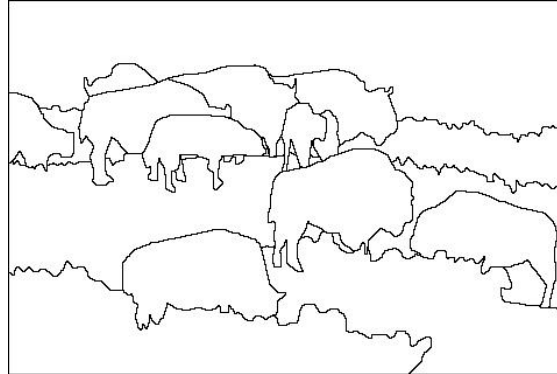
J. Canny, [***A Computational Approach To Edge Detection***](#), IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986.

Edge detection is just the beginning...

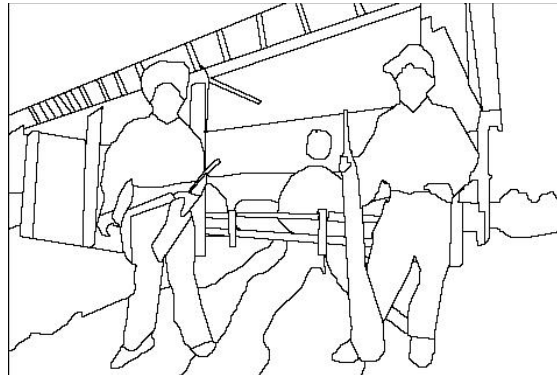
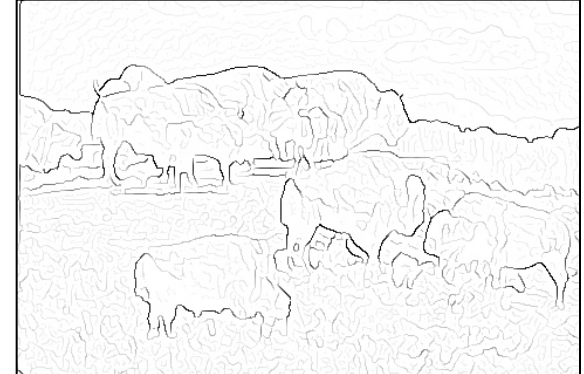
image



human segmentation



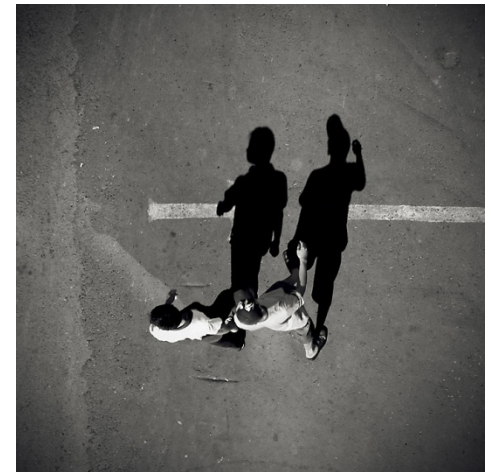
gradient magnitude



Berkeley segmentation database:

<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/>

Low-level edges vs. perceived contours



Background

Texture

Shadows